

UNIT III : Analytical Questions

Slot C

Name : K. Lahari

Course : Cryptography And Network Security

Reg No. : 192325118

Course Code : CSA5104

1. MD5

Problem 1 – MD5 Padding and Block Calculation

Question:

1. Convert the message "HELLO" into ASCII hexadecimal.
2. Calculate the original message length in bits.
3. Apply MD5 padding.
4. Append the 64-bit length field.
5. Determine the total number of 512-bit blocks after padding.

Step-by-Step Answer:

- ASCII hex of "HELLO": H=48, E=45, L=4C, L=4C, O=4F → 48 45 4C 4C 4F
- Length: "HELLO" = 5 characters = 5 bytes = 40 bits.
- MD5 padding rule: append a single '1' bit, then '0' bits, until the length $\equiv 448 \pmod{512}$. Here 40 bits + padding = 448 bits, so 408 padding bits are added (1 one-bit followed by 407 zero-bits).
- Append the 64-bit original length field (value = 40, i.e. 0x0000000000000028) → 448 + 64 = 512 bits total.
- Total blocks = 512 bits ÷ 512 bits/block = 1 block.

Final Answer: The padded message occupies exactly 1 MD5 block (512 bits).

Problem 2 – MD5 Compression Analysis (F function)

Question:

1. Given A=67452301, B=EFCDB89, C=98BADCFE, D=10325476.
2. $F(B,C,D) = (B \text{ AND } C) \text{ OR } ((\text{NOT } B) \text{ AND } D)$.
3. Calculate $F(B,C,D)$ and explain how the result is used in the first MD5 round.

Step-by-Step Answer:

Step 1 – Compute B AND C

B = EFCDB89 = 1110 1111 1100 1101 1010 1011 1000 1001

C = 98BADCFE = 1001 1000 1011 1010 1101 1100 1111 1110

B AND C = 88888888 (each nibble ANDs down to 1000 = 8)

Step 2 – Compute NOT B

NOT B = NOT(EFCDB89) = 10325476 (this happens to equal D exactly)

Step 3 – Compute (NOT B) AND D

(NOT B) AND D = 10325476 AND 10325476 = 10325476

Step 4 – OR the two results

$F(B,C,D) = 88888888 \text{ OR } 10325476 = 98\text{BADCFE}$

Result: $F(B,C,D) = 98\text{BADCFE}$

How it is used / why it provides nonlinear mixing:

- In round 1, $F(B,C,D)$ acts as a bitwise selector (multiplexer): for every bit position where $B=1$ it takes the bit from C , and where $B=0$ it takes the bit from D .
- The result is combined as: $A = B + \text{ROTL}_s(A + F(B,C,D) + M[k] + T[i])$, then A, B, C, D are rotated among the four registers for the next step.
- Because F depends on AND/OR/NOT rather than simple addition or XOR alone, its output is not a linear function of its inputs — a single input bit flip does not predictably flip a fixed set of output bits. This nonlinearity is essential to resist linear and differential cryptanalysis, ensuring the compression function cannot be approximated or inverted by simple algebraic means.

Problem 3 – MD5 Security Analysis

Question:

1. Two different messages $M1$ and $M2$ are found to produce (effectively) the same MD5 digest.
2. What security property has been violated?
3. Is MD5 suitable for digital signatures?
4. Compare collision resistance, preimage resistance, and second-preimage resistance.
5. Explain why MD5 is considered insecure today.

Step-by-Step Answer:

- Property violated: Collision resistance. Two distinct inputs $M1 \neq M2$ producing the same hash output is precisely the definition of a hash collision.
- Suitability for digital signatures: No. A digital signature scheme signs the hash of a document, not the document itself. If an attacker can find two different documents with the same MD5 hash, they can get one document signed and then substitute the other while the signature still "verifies" — allowing forgery. MD5 must not be used for signatures.
- Comparison of the three properties: Collision resistance = hard to find ANY two inputs (x, y) , $x \neq y$, with $\text{hash}(x) = \text{hash}(y)$. Preimage resistance = given a hash h , hard to find ANY input x with $\text{hash}(x) = h$ (one-wayness). Second-preimage resistance = given a SPECIFIC input $x1$, hard to find a different $x2$ with $\text{hash}(x2) = \text{hash}(x1)$. For an n -bit hash, ideal strength is $2^{(n/2)}$ for collision resistance (birthday bound) and 2^n for preimage/second-preimage resistance. For MD5 ($n=128$) collision resistance has been fully broken (collisions found in seconds), while preimage resistance, though weakened by cryptanalysis, remains closer to its theoretical bound.
- Why MD5 is insecure today: Its compression function has structural weaknesses that allow efficient differential-cryptanalysis attacks (Wang et al., 2004) to construct colliding message blocks in well under a second on ordinary hardware. Combined with only a 128-bit digest (weak by modern standards), MD5 is unsuitable for any security-critical application — it remains usable only for non-security purposes such as checksums for accidental corruption.

2. SHA-1

Problem 1 – SHA-1 Message Padding

Question:

1. Convert the message "ABC" into binary/hexadecimal.
2. Determine its length in bits.
3. Perform SHA-1 padding.
4. Determine the size of the padded message.
5. Explain why the final 64 bits contain the original message length.

Step-by-Step Answer:

- "ABC" in ASCII hex: A=41, B=42, C=43 → 41 42 43. In binary: 01000001 01000010 01000011.
- Length = 3 bytes = 24 bits.
- SHA-1 padding (identical scheme to MD5): append a single '1' bit, then '0' bits until length $\equiv 448 \pmod{512}$. $24 + \text{padding} = 448 \rightarrow 424$ padding bits added (1 one-bit + 423 zero-bits).
- Padded message size = 448 bits (message + padding) + 64 bits (length field) = 512 bits = exactly 1 block.
- Why the last 64 bits store the original length: it removes any ambiguity about where the real message ends and the padding begins (since padding could otherwise look like part of the message), and it also binds the hash to a specific message length, which blocks trivial padding/forgery tricks and is required for the compression function's Merkle–Damgård construction to be secure against length-related attacks.

Problem 2 – SHA-1 Word Expansion

Question:

1. $W_t = \text{ROTL1}(W_{t-3} \text{ XOR } W_{t-8} \text{ XOR } W_{t-14} \text{ XOR } W_{t-16})$.
2. Given $W_0=12345678$, $W_2=ABCDEF01$, $W_8=11111111$, $W_{13}=22222222$.
3. Calculate W_{16} (which needs W_{13} , W_8 , W_2 , W_0) showing the XOR and rotation steps.

Step-by-Step Answer:

For $t = 16$: $W_{16} = \text{ROTL1}(W_{13} \text{ XOR } W_8 \text{ XOR } W_2 \text{ XOR } W_0)$.

Step 1 – $W_{13} \text{ XOR } W_8$

$22222222 \text{ XOR } 11111111 = 33333333$

Step 2 – XOR with W_2

$33333333 \text{ XOR } ABCDEF01 = 98FEDC32$

Step 3 – XOR with W_0

$98FEDC32 \text{ XOR } 12345678 = 8ACA8A4A$

Step 4 – ROTL1 (rotate left by 1 bit)

$8ACA8A4A$ in binary = 1000 1010 1100 1010 1000 1010 0100 1010

Shift all bits left by 1, wrap the leading bit (1) to the end:

→ 0001 0101 1001 0101 0001 0100 1001 0101 = 15951495

Result: $W_{16} = 15951495$

Problem 3 – SHA-1 Security Analysis

Question:

1. A system uses SHA-1 to generate integrity values for downloaded software. An attacker discovers a practical collision attack against SHA-1.
2. Explain how a collision can affect the system.
3. Which security property of SHA-1 has failed?
4. Why is SHA-256 preferred?
5. Would simply increasing the SHA-1 output storage from 160 bits to 320 bits solve the problem? Explain.

Step-by-Step Answer:

- Impact of a collision: an attacker can craft a malicious software package whose SHA-1 hash matches the hash of the legitimate package. Since the integrity check will show a "match", users (or auto-update systems) would accept the malicious file as authentic, defeating the entire purpose of the integrity check.
- Property that failed: collision resistance (an efficient method — e.g. the 2017 "SHattered" attack — exists to find two different inputs with the same SHA-1 digest, far faster than the ideal 2^{80} birthday-bound effort).
- Why SHA-256 is preferred: it has a larger 256-bit digest (ideal collision resistance $\approx 2^{128}$ versus SHA-1's 2^{80}), a more thoroughly analyzed compression function, and — critically — no known practical collision or preimage attacks as of today, making it the current standard for integrity and signing.
- Would 320-bit storage fix SHA-1? No. Simply storing more bits (e.g. zero-padding the 160-bit digest out to 320 bits) does not add any real entropy or strengthen the underlying compression function — the actual collision-finding attack still only needs to break the original 160-bit SHA-1 algorithm. Security strength comes from the algorithm's internal design and true digest entropy, not from how many bits are allocated for storage.

3. SHA-224

Problem 1 – Output Size Analysis (SHA-224 vs SHA-256)

Question:

1. Calculate the difference in digest size.
2. Calculate the number of hexadecimal characters in each digest.
3. Calculate the approximate ideal collision resistance for each.
4. Calculate the approximate ideal preimage resistance for each.

Step-by-Step Answer:

Algorithm	Digest	Hex characters	Ideal collision resistance	Ideal preimage resistance
SHA-224	224 bits	56 ($224 \div 4$)	2^{112}	2^{224}
SHA-256	256 bits	64 ($256 \div 4$)	2^{128}	2^{256}

Difference in digest size: $256 - 224 = 32$ bits. Collision resistance follows the birthday bound (half the digest size in the exponent), while preimage resistance is ideally the full digest size.

Problem 2 – Padding Analysis (1000-bit message)

Question:

1. Determine the number of 512-bit blocks required before padding.
2. Determine the number of padding bits required.
3. Determine the number of final blocks after appending the 64-bit length field.
4. Explain why SHA-224 may require an additional block.

Step-by-Step Answer:

- Blocks needed just to hold the raw 1000-bit message: $\text{ceil}(1000 \div 512) = 2$ blocks.
- Total padded length must be the smallest multiple of 512 that is $\geq 1000 + 1$ (the mandatory '1' bit) + 64 (length field) = 1065 bits. The smallest such multiple is 1536 bits (3×512). Zero-padding bits required = $1536 - 1000 - 1 - 64 = 471$ zero bits (plus the single leading '1' bit).
- Final number of blocks after appending the 64-bit length field = $1536 \div 512 = 3$ blocks.
- Why an extra block may be needed: $1000 \bmod 512 = 488$ bits are already used in the current block. Since $488 + 1$ (the '1' bit) = 489 exceeds 448, there isn't enough room left in that block for the '1' bit, zero padding, and the 64-bit length field to fit within the same block — so the padding and length field must spill into an entirely new (third) block.

Problem 3 – SHA-224 vs SHA-256 (Algorithm Selection)

Question:

1. An organization requires a hash with at least ideal collision resistance but wants a shorter digest.
2. Determine whether SHA-224 or SHA-256 is more appropriate, justifying using digest size, collision resistance, storage requirements, and security margin.

Step-by-Step Answer:

- Digest size: SHA-224 (224 bits) is shorter than SHA-256 (256 bits), directly saving storage/bandwidth per hash value stored (56 vs 64 hex characters).
- Collision resistance: SHA-224 offers $\sim 2^{112}$ ideal collision resistance, which is still comfortably above the commonly accepted modern security floor (2^{112} is NIST's minimum acceptable strength through 2030 and beyond for many use cases).
- Storage requirements: lower for SHA-224 — meaningful at scale (e.g., large databases of file hashes or blockchain-style ledgers).
- Security margin: SHA-256 has a larger margin (2^{128}) and is more common/more analyzed, so if long-term (post-2030) or very high-assurance security matters more than saving a few bytes, SHA-256 is safer.

Conclusion:

If the requirement is strictly "at least ideal (≥ 112 -bit) collision resistance with the shortest possible digest," SHA-224 satisfies the requirement and is the more space-efficient choice. If maximum long-term security margin is the priority, SHA-256 remains the safer general-purpose recommendation.

4. SHA-256

Problem 1 – SHA-256 Padding

Question:

1. Find the length of "HELLO WORLD" in bits.
2. Perform SHA-256 padding.
3. Append the 64-bit length.
4. Determine the number of 512-bit blocks.

Step-by-Step Answer:

- "HELLO WORLD" = 11 characters (including the space) = 11 bytes = 88 bits.
- Padding: append a '1' bit then '0' bits until the length $\equiv 448 \pmod{512}$. $88 + \text{padding} = 448 \rightarrow 360$ padding bits (1 one-bit + 359 zero-bits).
- Append the 64-bit length field (value = 88) $\rightarrow 448 + 64 = 512$ bits total.
- Total blocks = $512 \div 512 = 1$ block. The entire padded message fits within a single SHA-256 block.

Problem 2 – SHA-256 Σ_0 Function

Question:

1. $x = 6A09E667$.
2. $\Sigma_0(x) = \text{ROTR}_2(x) \text{ XOR } \text{ROTR}_{13}(x) \text{ XOR } \text{ROTR}_{22}(x)$.
3. Show $\text{ROTR}_2(x)$, $\text{ROTR}_{13}(x)$, $\text{ROTR}_{22}(x)$, and the final XOR result.
4. Why does SHA-256 use rotations and XOR instead of simple shifting?

Step-by-Step Answer:

$x = 6A09E667 = 0110\ 1010\ 0000\ 1001\ 1110\ 0110\ 0110\ 0111$

$\text{ROTR}_2(x)$: rotate every bit 2 positions to the right (wrap around)

$\text{ROTR}_2(x) = \text{DA827999}$

$\text{ROTR}_{13}(x)$: rotate every bit 13 positions to the right

$\text{ROTR}_{13}(x) = 333B504F$

$\text{ROTR}_{22}(x)$: rotate every bit 22 positions to the right

$\text{ROTR}_{22}(x) = 27999DA8$

Final XOR: $\text{ROTR}_2(x) \text{ XOR } \text{ROTR}_{13}(x) \text{ XOR } \text{ROTR}_{22}(x)$

$\text{DA827999 XOR } 333B504F = \text{E9B929D6}$

$\text{E9B929D6 XOR } 27999DA8 = \text{CE20B47E}$

Result: $\Sigma_0(x) = \text{CE20B47E}$

Why rotation + XOR instead of a simple shift:

- A logical shift (SHR) discards bits that fall off the end, permanently losing information and reducing entropy each time it's applied.
- A rotation (ROTR) preserves every bit of the original value — it only repositions them — so no information is ever lost, keeping the transformation reversible and preserving full entropy.
- XOR-combining three differently-rotated copies of the same word spreads (diffuses) the influence of every input bit across many output bit positions nonlinearly, which strengthens resistance to linear and differential cryptanalysis and produces a strong avalanche effect.

Problem 3 – SHA-256 vs MD5 (Banking Application)**Question:**

1. A banking application currently uses MD5 for password-related integrity checking.
2. Compare MD5 and SHA-256 on digest length, word size, number of rounds, collision resistance, and current security status.
3. Recommend the appropriate algorithm and justify.

Step-by-Step Answer:

Feature	MD5	SHA-256
Digest length	128 bits	256 bits
Word size	32 bits	32 bits
Number of rounds	64	64
Collision resistance	Broken in practice	$\approx 2^{128}$, no known practical attack
Current security status	Cryptographically broken — deprecated	Considered secure / current standard

Recommendation:

Migrate to SHA-256 (and ideally use it inside a proper password-hashing construction such as PBKDF2-HMAC-SHA256, bcrypt, or Argon2 rather than a single raw hash — see Section 8, Problem 2). MD5 must be retired immediately for any security-relevant integrity or password-related function because its collision resistance is fully broken, exposing the bank to forgery and integrity-bypass risks.

5. SHA-384**Problem 1 – SHA-384 Output Calculation****Question:**

1. A SHA-384 digest is given as a hexadecimal string containing 96 hexadecimal characters.
2. Determine the number of bits, number of bytes, compare with SHA-256, and explain the advantage of the longer digest.

Step-by-Step Answer:

- Number of bits = 96 hex characters $\times$ 4 bits/character = 384 bits.
- Number of bytes = $384 \div 8 = 48$ bytes.
- Comparison with SHA-256: SHA-256 produces a 256-bit / 32-byte / 64-hex-character digest, so SHA-384's digest is 128 bits (16 bytes / 32 hex characters) longer.
- Advantage of the longer digest: a larger output gives a higher ideal collision resistance ($\approx 2^{192}$ vs $\approx 2^{128}$) and preimage resistance ($\approx 2^{384}$ vs $\approx 2^{256}$), providing a bigger security margin — useful for long-term archival integrity or high-assurance/government-grade applications, at the cost of larger storage and slightly more computation.

Problem 2 – SHA-384 Block Analysis (1000-byte message)

Question:

1. A message contains 1000 bytes. Determine the message size in bits, number of 1024-bit blocks required after padding, why SHA-384 uses 1024-bit blocks, and how its processing structure compares with SHA-256.

Step-by-Step Answer:

- Message size = 1000 bytes $\times$ 8 = 8000 bits.
- Padded total must be the smallest multiple of 1024 $\geq 8000 + 1$ (one-bit) + 128 (SHA-384's length field is 128 bits, not 64) = 8129 bits. The smallest multiple of 1024 that satisfies this is 8192 bits (8×1024). So 8 blocks are required.
- Why a 1024-bit block: SHA-384 operates on sixteen 64-bit words per block ($16 \times 64 = 1024$ bits), matching its 64-bit internal word size. This larger block/word size improves throughput on modern 64-bit CPUs and allows a 128-bit length field, supporting astronomically large messages.
- Comparison with SHA-256: SHA-384 uses 64-bit words / 1024-bit blocks / 80 rounds versus SHA-256's 32-bit words / 512-bit blocks / 64 rounds. Internally SHA-384 is simply SHA-512 run with different initial hash values, with the final 512-bit output truncated to 384 bits.

Problem 3 – SHA-384 Security Analysis ("Twice as secure" claim)

Question:

1. A security engineer says: "SHA-384 is always twice as secure as SHA-256 because $384 > 256$."
2. Analyze this statement using collision resistance, preimage resistance, digest size, computational cost, and whether "twice as secure" is technically correct.

Step-by-Step Answer:

- Digest size: 384 bits is indeed $1.5\times$ larger than 256 bits (not "twice").
- Collision resistance: SHA-384 $\approx 2^{192}$ vs SHA-256 $\approx 2^{128}$ — that is a difference of 2^{64} , an enormous (exponential) increase, not a simple "2 $\times$ " multiple.
- Preimage resistance: SHA-384 $\approx 2^{384}$ vs SHA-256 $\approx 2^{256}$ — again an exponential, not linear, difference.
- Computational cost: SHA-384 requires more computation per byte (larger words/blocks, more rounds) than SHA-256, so the extra security comes at a real performance cost.
- Is "twice as secure" technically correct? No. Security strength in cryptography is measured in bits of an exponent (2^n), so comparing digest sizes as a linear ratio ($384/256 = 1.5\times$) is meaningless — the actual security gap is exponential, not linear. The statement conflates digest length with security strength, which is an oversimplification.

Conclusion:

The engineer's statement is technically incorrect. Security strength does not scale linearly with digest length; the real difference between SHA-256 and SHA-384 is an exponential increase in resistance to brute-force/collision attacks, not a fixed "2×" factor.

6. SHA-512

Problem 1 – SHA-512 Padding ($L = 900$ bits)

Question:

1. Determine the number of 1024-bit blocks before padding.
2. Determine the padding requirements.
3. Determine the length-field size.
4. Determine the final number of blocks.

Step-by-Step Answer:

- Blocks needed to hold the raw 900-bit message: $\text{ceil}(900 \div 1024) = 1$ block.
- Padding requirement: total padded length must be the smallest multiple of 1024 that is $\geq 900 + 1$ (one-bit) + 128 (length field) = 1029. The smallest multiple of 1024 satisfying this is 2048 bits (2×1024), since 1024 itself (1 block) is smaller than 1029. Zero-padding bits required = $2048 - 900 - 1 - 128 = 1019$ zero bits (plus the leading '1' bit).
- Length-field size: SHA-512 uses a 128-bit length field (unlike SHA-256's 64-bit field), reflecting its 1024-bit block design and support for extremely long messages.
- Final number of blocks = $2048 \div 1024 = 2$ blocks.

Problem 2 – SHA-512 Word Operation

Question:

1. $x = 6A09E667F3BCC908$ (64-bit word).
2. $\Sigma_0(x) = \text{ROTR1}(x) \text{ XOR } \text{ROTR8}(x) \text{ XOR } \text{SHR7}(x)$.
3. Show all intermediate operations, and explain how 64-bit words affect processing compared with SHA-256.

Step-by-Step Answer:

$x = 6A09E667F3BCC908$

ROTR8(x) — rotate right by 8 bits = move the last byte to the front:

$\text{ROTR8}(x) = 086A09E667F3BCC9$

ROTR1(x) — rotate right by 1 bit (x is even, so no wrap bit is set):

$\text{ROTR1}(x) = 3504F333F9DE6484$

SHR7(x) — logical shift right by 7 bits (top 7 bits become 0, bottom 7 bits discarded):

$\text{SHR7}(x) = 00D413CCCFE77992$

XOR all three together:

$\text{ROTR1}(x) \text{ XOR } \text{ROTR8}(x) = 3D6EFAD59E2DD84D$

Result $\text{XOR SHR7}(x) = 3\text{DBAE91951CAA1DF}$

Result: $\Sigma 0(x) = 3\text{DBAE91951CAA1DF}$

Effect of 64-bit words vs SHA-256's 32-bit words:

- Each SHA-512 word carries twice the data per operation, so a 1024-bit block is processed in the same number of "word operations" as a 512-bit SHA-256 block, giving higher throughput on modern 64-bit CPUs that natively handle 64-bit arithmetic in one cycle.
- Larger words also allow larger rotation constants and a bigger internal state, contributing to a larger security margin, at the cost of being less efficient on 32-bit or embedded processors.

Problem 3 – SHA-512 vs SHA-256 (Large-Data Server)

Question:

1. A server must process very large amounts of data. Compare SHA-256 and SHA-512 on word size, block size, digest size, number of rounds, performance on 64-bit processors, and security strength. Recommend one algorithm and justify.

Step-by-Step Answer:

Feature	SHA-256	SHA-512
Word size	32 bits	64 bits
Block size	512 bits	1024 bits
Digest size	256 bits	512 bits
Rounds	64	80
Performance on 64-bit CPUs	Good, but under-uses 64-bit registers	Better — native 64-bit word operations
Security strength	$\approx 2^{128}$ collision resistance	$\approx 2^{256}$ collision resistance

Recommendation:

For a modern 64-bit server handling large volumes of data, SHA-512 is generally the better choice: its 64-bit-word design maps efficiently onto 64-bit CPU registers/instructions, often processing large data faster than SHA-256 despite doing more "rounds," while also providing a substantially larger security margin. (If a shorter 256-bit output is specifically required while keeping this performance advantage, SHA-512/256 — see Section 7 — is an excellent alternative.

7. SHA-512/224 and SHA-512/256

Problem 1 – Digest Comparison

Question:

1. Compare SHA-224, SHA-256, SHA-512/224, SHA-512/256.
2. Determine which produce a 224-bit digest, which produce a 256-bit digest, and which use 512-bit internal processing.
3. Explain why SHA-512/224 can produce a 224-bit digest while internally using SHA-512.

Step-by-Step Answer:

- 224-bit digest: SHA-224 and SHA-512/224.
- 256-bit digest: SHA-256 and SHA-512/256.

- 512-bit internal processing (1024-bit blocks, 64-bit words, SHA-512 compression function): SHA-512/224 and SHA-512/256.

Why SHA-512/224 gives a 224-bit output despite internal SHA-512 processing:

SHA-512/224 runs the full SHA-512 algorithm (1024-bit blocks, 64-bit words, 80 rounds) but starts from a different, specially-generated set of initial hash values (as defined in FIPS 180-4) and then truncates the final 512-bit internal digest down to its leftmost 224 bits. This gives the speed/architecture benefits of SHA-512 while producing a compact 224-bit output, and the distinct initial values also make the result cryptographically distinct from (and not derivable from) plain SHA-512 or SHA-512/256 output.

Problem 2 – Security Analysis (224-bit hash on a 64-bit processor)

Question:

1. A company requires a 224-bit hash output but operates mainly on 64-bit processors.
2. Which is potentially more suitable: SHA-224 or SHA-512/224? Compare internal word size, block size, digest size, security level, and processing environment.

Step-by-Step Answer:

Aspect	SHA-224	SHA-512/224
Internal word size	32 bits	64 bits
Block size	512 bits	1024 bits
Digest size	224 bits	224 bits
Security level	$\approx 2^{112}$ (comparable)	$\approx 2^{112}$ (comparable)
Best processing environment	32-bit / embedded systems	64-bit servers / modern CPUs

Conclusion:

Since the company operates mainly on 64-bit processors, SHA-512/224 is potentially more suitable — it delivers the same 224-bit output and comparable security level as SHA-224, but its 64-bit-word operations run more efficiently on 64-bit hardware.

Problem 3 – Algorithm Selection ("SHA-512/256 is unnecessary" claim)

Question:

1. A developer proposes: "SHA-512/256 is unnecessary because SHA-256 already produces a 256-bit digest."
2. Analyze the statement and explain why SHA-512/256 may still be useful.

Step-by-Step Answer:

- Performance: SHA-512/256 uses 64-bit word operations, which can run faster than SHA-256's 32-bit operations on modern 64-bit server hardware, despite producing the same 256-bit digest length.
- Length-extension resistance: SHA-512/256 (like SHA-224 and SHA-384) is truncated from a larger internal state using distinct initial values, which — per FIPS 180-4 — makes it inherently resistant to length-extension attacks, a weakness that plain SHA-256 and SHA-512 both have.
- Larger internal state: even though the final output is 256 bits, the internal 512-bit processing state can offer extra resilience against certain internal structural attacks.

Conclusion:

The developer's statement is an oversimplification — matching digest length does not make the two algorithms equivalent in performance or in resistance to length-extension attacks. SHA-512/256 remains a useful option, particularly on 64-bit server infrastructure or wherever length-extension resistance is a requirement.

8. SHA Version Comparison

Problem 1 – Complete Comparison Table

Question:

1. Complete the comparison table for digest size, word size, block size, number of rounds, and security status across MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512.

Step-by-Step Answer:

Feature	MD5	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Digest size	128 bit	160 bit	224 bit	256 bit	384 bit	512 bit
Word size	32 bit	32 bit	32 bit	32 bit	64 bit	64 bit
Block size	512 bit	512 bit	512 bit	512 bit	1024 bit	1024 bit
Rounds	64	80	64	64	80	80
Security status	Broken	Broken (practical collisions)	Secure	Secure	Secure	Secure

Problem 2 – Algorithm Selection Scenario

Question:

1. Select an appropriate modern hashing approach for: (A) File integrity, (B) Digital signatures, (C) Password storage, (D) Blockchain-style data integrity, and explain why MD5 and SHA-1 should not be selected for new security applications.

Step-by-Step Answer:**A. File integrity:**

SHA-256 (or SHA-3-256) — fast, strong collision resistance, widely supported by OS and package-manager checksum tools.

B. Digital signatures:

SHA-256 or SHA-384/512, matched to the signature algorithm's requirements (e.g., ECDSA/RSA-PSS commonly pair with SHA-256 or stronger) — strong collision resistance is essential since a signature is only as strong as the underlying hash.

C. Password storage:

Not a bare general-purpose hash at all — use a dedicated, deliberately slow password-hashing function such as Argon2, bcrypt, or PBKDF2-HMAC-SHA256, always with a unique per-user salt. General-purpose hashes like MD5/SHA-1/SHA-256 alone are far too fast, making brute-force and rainbow-table attacks practical.

D. Blockchain-style data integrity:

SHA-256 (as used by Bitcoin) or SHA-3 — needs strong, well-analyzed collision resistance because chained hashes make the whole ledger's integrity depend on it.

Why MD5 and SHA-1 should not be used for new applications:

Both have demonstrated practical collision attacks (MD5 trivially and instantly; SHA-1 via the 2017 "SHAttered" attack and subsequent chosen-prefix collision techniques), meaning an adversary can deliberately construct two different inputs sharing the same hash. This breaks the core security guarantee needed for integrity checks and signatures, and both algorithms have been formally deprecated by NIST and major software vendors.

Problem 3 – Hash Security Analysis Against Requirements

Question:

1. Requirements: Collision resistance $\geq 2^{120}$, Preimage resistance $\geq 2^{250}$, Digest ≤ 256 bits.
2. Evaluate whether MD5, SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512 satisfy the requirements, giving a reason for accepting/rejecting each.

Step-by-Step Answer:

Algorithm	Collision resistance	Preimage resistance	Digest ≤ 256 bits?	Verdict
MD5	2^{64} (practically 0)	2^{128}	Yes (128)	REJECT — collision & preimage both fail; also practically broken
SHA-1	2^{80} (practically broken)	2^{160}	Yes (160)	REJECT — collision resistance fails ($< 2^{120}$) and is practically broken
SHA-224	2^{112}	2^{224}	Yes (224)	REJECT — collision resistance $2^{112} < \text{required } 2^{120}$
SHA-256	2^{128}	2^{256}	Yes (256)	ACCEPT — meets all three requirements
SHA-384	2^{192}	2^{384}	No (384)	REJECT — digest size exceeds the 256-bit limit
SHA-512	2^{256}	2^{512}	No (512)	REJECT — digest size exceeds the 256-bit limit

Conclusion:

Only SHA-256 satisfies all three stated requirements simultaneously — sufficient collision resistance ($\geq 2^{120}$), sufficient preimage resistance ($\geq 2^{250}$), and a digest no larger than 256 bits.